Lab 2: Hands-On Git and Version Control

Solution Document

Student GitHub	amnajaved-dev1
Terminal prompt	amnajaved-dev1@ubuntu:~$
Submission repo	DevOps
Directory	Labs/Lab02
Practice repo	DevOps_Lab2_Practice

This document explains the commands used for each task and the reason behind them. Screenshots are stored in **Labs/Lab02/screenshots/** with the exact required file names.

Getting Started: Install Git

```
git --version
```

Git is installed from git-scm.com. The version output confirms installation.

Evidence: screenshots/git_installation.png

Task 1: Create Private GitHub Repository

On GitHub: New repository, name **DevOps_Lab2_Practice**, visibility **Private**, then Create.

Evidence: screenshots/repo_private.png

Task 2: Connect Repository via SSH

```
ssh-keygen -t ed25519 -C "your_email@example.com"
cat ~/.ssh/id_ed25519.pub
git clone git@github.com:amnajaved-dev1/DevOps_Lab2_Practice.git
```

The public key (.pub) is copied to GitHub under Settings > SSH and GPG keys > New SSH key. The private key is never shown or shared. SSH lets Git authenticate without typing a password each time.

Evidence: screenshots/ssh_keygen.png, github_sshkey.png, ssh_clone.png

Task 3: Configure Git Username and Email

```
git config --global user.name "Your Name"
git config --global user.email "your_email@example.com"
git config --list
```

Every commit stores the author name and email, so this links commits to you.

Evidence: screenshots/git_identity.png, git_config_list.png

Task 4: Explore the .git Folder

```
cd DevOps_Lab2_Practice
ls -a .git
```

The .git folder holds the full repository: **HEAD** (current branch pointer), **config** (repo settings), **objects/** (commits, trees, blobs), **refs/** (branches and tags), **hooks/**, and **index** (staging area).

Evidence: screenshots/git_folder.png

Task 5: Local Repository Management

```
rm -rf .git
git status
```

Why the error? `git status` prints *fatal: not a git repository (or any of the parent directories): .git*. Git identifies a repository by the .git folder. After deleting it, the folder is a plain directory with no history or tracking, so Git commands fail.

Evidence: screenshots/delete_git.png

```
git init
echo "# Lab02 Git Practice" >> README.md
git add README.md
git commit -m "Initial commit"
git remote add origin git@github.com:amnajaved-dev1/DevOps_Lab2_Practice.git
git branch -M main
git push -u origin main
```

git init creates a new empty repository. **git add** stages the file, **git commit** records it. **git remote add origin** links to GitHub. **git branch -M main** renames the branch to main. **push -u** uploads and sets the upstream branch.

Evidence: screenshots/git_init.png, first_commit.png, first_push.png

Task 6: File Status and Staging

```
echo "Adding some test notes" >> notes.txt
git status
git add notes.txt
git commit -m "Add notes.txt"
```

Before staging, status shows notes.txt as **untracked**. After *git add* it is **staged** (Changes to be committed). After the commit the working tree is clean. Editing notes.txt again makes it **modified**, and the same add/commit cycle is repeated.

Evidence: screenshots/status1.png, commit_notes.png

Task 7: Branch Creation Using GitHub GUI

```
git pull origin bugfix/user-auth-error
git branch -a
```

The branch **bugfix/user-auth-error** is created from the branch dropdown on GitHub, then fetched locally with pull.

Evidence: screenshots/bugfix_branch_gui.png, bugfix_branch_local.png

Task 8: Branch Creation and Push Using Git Bash

```
git checkout -b feature/db-connection
git push origin feature/db-connection
```

checkout -b creates and switches to a new branch. Push publishes it to the remote.

Evidence: screenshots/feature_db_branch.png

Task 9: Branching and Merging

```
git checkout -b feature-1
# add a function to main.py, e.g.
# def greet(name): return "Hello, " + name
git add main.py
```

```
git commit -m "Add new function to main.py"
git checkout main
git merge feature-1
git push origin main
git push origin feature-1
```

Work is done on an isolated branch, then merged into main. In real teams a Pull Request is preferred over direct local merges because it enables review.

Evidence: screenshots/branch_create.png, feature_commit.png, merge.png, push_branches.png

Task 10: Pull Request and Branch Review (GitHub GUI)

1. Open Pull requests > New pull request, base: **main**, compare: **feature/db-connection**.
2. Review the Files changed tab, then click **Merge pull request** and confirm.
3. Click **Delete branch** after the merge.

Evidence: screenshots/pr_creation.png, pr_merge.png, branch_delete.png

Task 11: Detailed Branch Strategy

```
git checkout main
git branch develop
git branch staging
git checkout -b feature/login develop
# edit, then:
git add . && git commit -m "Add login feature"
git checkout develop && git merge feature/login
git checkout staging && git merge develop
git checkout main && git merge staging
git branch -a
git push origin develop staging main
```

Flow: **feature/* and bugfix/*** to **develop** to **staging** (testing) to **main** (production). Each stage adds a quality gate before production.

Evidence: screenshots/branch_strategy.png, branch_merges.png, final_merge.png

Task 12: Code Review Workflow

Create PR: from a feature branch into main with a clear title and description. **Assign reviewer:** use the Reviewers panel on the right.
Reviewer actions: Files changed > Review changes > *Request changes* with comments; close a separate practice PR with *Close pull request*.
Author: fix issues on the same branch and push; the PR updates automatically.

```
git add .
git commit -m "Address review comments"
git push origin feature/db-connection
```

Merge: after approval choose a merge style (merge commit, squash, or rebase), confirm, then delete the branch.

Evidence: screenshots/pr_create_details.png, pr_assigned_reviewer.png, pr_request_changes.png, pr_rejected.png, pr_updated_with_commits.png, pr_merge_confirm.png, pr_merged.png, pr_branch_deleted.png

Task 13: Branch Cleanup Best Practices

```
git push origin --delete <branch-name>
git checkout main
```

```
git pull origin main
git branch -d <branch-name>
git branch
```

Deleting merged branches keeps the repository tidy. **-d** is the safe delete (refuses if unmerged); **-D** forces deletion.

Evidence: screenshots/remote_branch_deleted.png, remote_branch_delete_cmd.png

Final Submission

```
git status
git add Labs/Lab02
git commit -m "Submit Lab 02 evidence"
git push origin main
```

All files (README.md, notes.txt, main.py, Lab2_Solution.pdf, screenshots/) are in **DevOps/Labs/Lab02** and verified visible on GitHub.